UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación

Carrera de Software

Modelado de comunicación entre procesos distribuidos

Escenario 3: Red social “Vecindario”

GA-TL-02 – FORM

Asignatura: Aplicaciones Distribuidas (ISR-701)

Docente:	Gleiston Cicerón Guerrero Ulloa
CALDERON SALTOS JOSEPH ALEXANDER	_____
GARCIA SANGOQUIZA ROGER STEVEN	_____
MERA JUMBO ALLISON MICHELLE	_____
ZAMBRANO PARRAGA LUIS MARIO	_____
Nivel/paralelo:	Séptimo / A
Fecha:	Noviembre de 2026

Quevedo – Ecuador

Índice

1. Introducción y alcance **2**

2. Actores y componentes **2**

3. Matriz de interacciones **2**

4. Justificación técnica por componente **3**

4.1. Publicaciones e imágenes 3

4.2. Muro y seguidores 5

4.3. Mensajería directa 7

4.4. Moderación y notificaciones 8

5. Requisitos del escenario y decisiones de diseño **11**

6. Situaciones de fallo **12**

7. Alternativas y limitaciones del diseño **13**

8. Preguntas críticas **14**

9. Conclusiones **15**

10. Declaración de uso de inteligencia artificial **15**

Referencias **16**

1. Introducción y alcance

Vecindario es el escenario ficticio de una red social para comunidades barriales de Ecuador. El equipo debe modelar cómo intercambian mensajes sus procesos al crear publicaciones, distribuir avisos, conversar en tiempo real, mostrar la presencia de usuarios y atender denuncias. Los requisitos y las situaciones de fallo provienen del Anexo A entregado por el docente; no son mediciones realizadas por el equipo.

Se propone separar las solicitudes que necesitan una respuesta inmediata de la distribución asíncrona de eventos. Esta decisión se examina en las interacciones de cada servicio, junto con sus ventajas y limitaciones.

Alcance de este documento. Presentar el diagrama de comunicación, describir las interacciones, justificar las tecnologías elegidas y registrar la exposición. Los valores de rendimiento del enunciado son objetivos de diseño; no deben presentarse como resultados comprobados sin ensayos.

2. Actores y componentes

Actores: usuario, negocio local y moderador.

Sistema externo: almacenamiento de imágenes.

Los siete procesos mínimos del escenario son: cliente (web o móvil), Publicaciones, Seguidores, Muro, Mensajería directa con presencia en línea, Moderación y Notificaciones.

La propuesta incorpora PostgreSQL y RabbitMQ como componentes de infraestructura para persistencia y entrega de eventos.

La separación entre procesos debe mantenerse visible en el diagrama editable.

3. Matriz de interacciones

Tabla 1: Matriz completa de interacciones.

ID	Origen → destino	Tipo de comunicación	Protocolo / tecnología	Sincronía
I01	Cliente → Publicaciones	Petición / respuesta REST	HTTPS; JSON; API REST	Síncrona
I02	Cliente → almacenamiento externo	Transferencia de archivo	HTTPS PUT; URL prefirmada	Síncrona
I03	Cliente ↔ Muro	Petición / respuesta REST	HTTPS; JSON; API REST	Síncrona
I04	Cliente ↔ Mensajería	Sockets mediante WebSocket	WSS sobre TCP/TLS; biblioteca `ws`; ACK de aplicación	Asíncrona bidireccional
I05	Cliente ↔ Moderación	Petición / respuesta REST	HTTPS sobre TCP/TLS; JSON; API REST	Síncrona

ID	Origen → destino	Tipo de comunicación	Protocolo / tecnología	Sincronía
I06	Moderación ↔ PostgreSQL de denuncias	Cliente / base de datos	TCP; protocolo PostgreSQL; controlador de BD	Síncrona
I07	Moderación ↔ Publicaciones	RPC unaria	gRPC; HTTP/2 sobre TCP/TLS; Protocol Buffers	Síncrona
I08	Publicaciones ↔ PostgreSQL	Cliente / base de datos	TCP; protocolo PostgreSQL; controlador de BD	Síncrona
I09	Publicaciones → RabbitMQ	Mensajería asíncrona	AMQP 0-9-1; RabbitMQ	Asíncrona
I10	RabbitMQ → Muro	Mensajería asíncrona	AMQP 0-9-1 sobre TCP; RabbitMQ; ACK manual	Asíncrona
I11	Muro ↔ Seguidores	RPC unaria paginada	gRPC; HTTP/2 sobre TCP/TLS; Protocol Buffers	Síncrona por lote
I12	Muro → RabbitMQ	Mensajería asíncrona	AMQP 0-9-1 sobre TCP; RabbitMQ; outbox del Muro; confirmaciones de publicación	Asíncrona
I13	RabbitMQ → Mensajería	Mensajería asíncrona	AMQP 0-9-1 sobre TCP; RabbitMQ; ACK manual	Asíncrona
I14	Mensajería ↔ PostgreSQL de mensajes	Cliente / base de datos	TCP; protocolo PostgreSQL; controlador de BD	Síncrona
I15	Mensajería → RabbitMQ	Mensajería asíncrona	AMQP 0-9-1 sobre TCP; RabbitMQ; outbox de Mensajería	Asíncrona
I16	RabbitMQ → Notificaciones	Mensajería asíncrona	AMQP 0-9-1 sobre TCP; RabbitMQ; ACK manual	Asíncrona
I17	Cliente ↔ Notificaciones	Petición / respuesta REST	HTTPS sobre TCP/TLS; JSON; API REST	Síncrona
I18	Muro ↔ PostgreSQL del Muro	Cliente / base de datos	TCP; protocolo PostgreSQL; controlador de BD	Síncrona

4. Justificación técnica por componente

4.1. Publicaciones e imágenes

Este apartado describe las interacciones I01, I02, I08 e I09 del diseño de Vecindario. El escenario fija los objetivos de latencia y las situaciones de fallo; la selección de tecnologías es una propuesta del equipo y no constituye una implementación ni una prueba de rendimiento. Las referencias seleccionadas corresponden exclusivamente a artículos científicos con DOI.

I01. Cliente y servicio de Publicaciones. Para crear un aviso, el cliente consulta primero una API REST por HTTPS y solicita una URL temporal de carga. Tras subir la imagen, envía una petición con el texto, tipo de aviso, referencia de imagen e identidad del autor, que debe verificarse en el servidor. Publicaciones responde con el identificador del aviso o con un error de

validación.

Se considera una comunicación síncrona porque cada solicitud espera la respuesta correspondiente. Los estudios sobre APIs REST en sistemas distribuidos y las evaluaciones de comunicación entre microservicios permiten fundamentar la elección de una interfaz HTTP para estas operaciones de petición y respuesta [1], [2].

La decisión no implica que REST sea siempre la opción de menor latencia.

I02. Envío directo de la imagen. El Cliente carga el archivo con HTTPS `PUT` hacia el almacenamiento externo mediante una URL prefirmada obtenida en I01.

Se propone esta separación para evitar que Publicaciones tenga que retransmitir el contenido binario de todas las imágenes.

Chen y colaboradores estudian específicamente las cargas directas desde usuarios hacia servicios de almacenamiento en la nube y advierten sobre fallos de autenticación, autorización y sincronización [3].

Arhrib y colaboradores presentan una arquitectura publicada que emplea URL prefirmadas para transferir imágenes directamente a almacenamiento de objetos [4].

Por ello, la propuesta exige validar la autorización, limitar la vigencia y alcance de la URL y confirmar que el objeto se cargó antes de asociarlo definitivamente al aviso.

Estas fuentes respaldan el mecanismo y sus riesgos, no demuestran que Vecindario ya lo tenga implementado.

I08. Persistencia del aviso y registro *outbox*. Publicaciones guarda en PostgreSQL tanto los datos del aviso como el registro de un evento pendiente.

Ambas escrituras se plantean dentro de una única transacción local: si una operación falla antes del *commit*, ninguna de las dos debe quedar confirmada.

La utilidad de la persistencia transaccional frente a fallos y eventos omitidos aparece en el estudio sobre consistencia de microservicios dirigidos por eventos de Bação y Guerreiro [5].

De forma más directa, Schwarz, Quast y Riehle describen un sistema real en que los microservicios escriben eventos en una tabla *outbox* y otro componente los entrega al intermediario de mensajería, precisamente para coordinar la escritura de datos con el envío de eventos [6].

La interacción I08 es síncrona mediante el protocolo PostgreSQL sobre TCP: el servicio espera el resultado de la transacción.

I09. Eventos asíncronos hacia RabbitMQ. Un despachador del servicio Publicaciones consulta los registros pendientes y publica mensajes `post.created` o `security.alert`, identificados por un ID único y por la publicación de origen.

En el diseño se utilizan AMQP y RabbitMQ; el cliente puede recibir la confirmación de registro sin esperar a que el Muro procese los eventos.

Kazanavičius y Mažeika comparan distintas tecnologías de comunicación entre microservicios, entre ellas HTTP REST y RabbitMQ, y distinguen decisiones síncronas y asíncronas [2].

Fu, Zhang y Yu comparan diferentes sistemas de colas, incluidos RabbitMQ, Kafka y Pulsar,

con criterios de rendimiento y características de diseño; sus resultados dependen del escenario, por lo que no fundamentan afirmar que RabbitMQ sea universalmente superior [7].

El diseño propone confirmaciones de publicación y reintentos. Estos últimos pueden generar duplicados: los consumidores deberán reconocer eventos ya procesados, coherentemente con el esquema de entrega al menos una vez descrito por Schwarz y colaboradores [6].

Relación con los requisitos R3 y el fallo F2. R3 exige que los avisos normales aparezcan en el Muro en un máximo de cinco segundos y las alertas de seguridad en menos de un segundo.

Se plantean rutas y consumidores separados para evitar que un volumen elevado de avisos normales retrase las alertas, pero esa decisión debe verificarse mediante pruebas antes de afirmar que cumple ambos límites.

Ante F2, si el Muro deja de funcionar, la combinación de eventos guardados y mensajería asíncrona busca impedir que se pierda la solicitud de distribución.

La persistencia y el procesamiento de duplicados son aspectos necesarios al discutir la consistencia entre servicios [5], [6].

Alternativa y costo de la decisión. La alternativa más simple sería guardar el aviso y publicar inmediatamente el evento en RabbitMQ, sin tabla *outbox*.

Sin embargo, si el proceso se detiene después de confirmar la base de datos y antes de publicar el evento, ambos sistemas quedarían descoordinados.

Se propone la tabla *outbox* para reducir ese riesgo; a cambio, habrá que implementar un despachador, reintentos y detección de duplicados [6].

4.2. Muro y seguidores

El servicio de Muro construye la línea de tiempo (*timeline*) de cada usuario a partir de los avisos publicados por las personas y negocios que este sigue. Su comunicación con el resto del sistema se resuelve mediante cinco interacciones (I03, I10, I11, I12 e I18), que combinan comunicación síncrona y asíncrona según la necesidad de cada operación.

I03. El cliente consulta su *timeline* mediante una petición HTTPS con API REST y formato JSON, enviando el identificador del usuario y un cursor de paginación. La comunicación es síncrona porque la persona espera ver su muro en pantalla. El Muro responde leyendo una proyección ya construida, en lugar de calcularla en el momento, y excluye los avisos marcados como ocultos.

I10. El Muro consume desde RabbitMQ los avisos normales, las alertas de seguridad y los eventos `post.hidden`. Se emplea mensajería asíncrona mediante AMQP 0-9-1, con colas durables y confirmación manual (*ACK*) del consumidor. El *ACK* se emite únicamente después de aplicar el cambio en la proyección, de modo que, si el servicio falla a mitad del procesamiento, el mensaje no se pierde y se reentrega.

Las rutas normal (`feed.normal`) y de alertas (`feed.alert`) se separan para que un lote grande de publicaciones comunes no retrase una alerta de seguridad, lo que responde al requisito R3.

I11. Para distribuir un aviso, el Muro necesita conocer quiénes siguen al autor. Esta consulta se resuelve con una llamada a procedimiento remoto (RPC) mediante gRPC sobre HTTP/2

y Protocol Buffers, de tipo unario y paginado: el Muro envía `ListFollowers(authorId, pageToken, pageSize)` y recibe una página de seguidores junto con el siguiente `nextToken`.

La comunicación es síncrona por lote, porque el Muro necesita cada página para continuar el procesamiento. La paginación evita enviar en un solo mensaje la lista completa de un negocio con 30 000 seguidores. Se descartó REST para esta comunicación entre servicios, ya que gRPC ofrece un contrato tipado y un formato binario. La literatura reciente compara de forma empírica ambos patrones de comunicación en arquitecturas de microservicios, en términos de latencia y rendimiento [8]; sin embargo, los resultados dependen del entorno de prueba y no se extrapolan directamente a este diseño.

I12. Cuando la proyección de un aviso está lista, el Muro publica un evento `feed.ready` (con el identificador de la publicación, la audiencia y el tipo) en RabbitMQ. Se usa un *outbox* del Muro y confirmaciones de publicación (*publisher confirms*), de modo que el evento se emita solo después de persistir la proyección. Posteriormente, el servicio de Mensajería lo consume para avisar a los clientes conectados.

I18. El Muro almacena en su propia base de datos PostgreSQL la proyección del *timeline*, el identificador de cada evento ya procesado (`eventId`) y el estado oculto de los avisos. La comunicación es síncrona mediante el protocolo nativo de PostgreSQL sobre TCP. Registrar el `eventId` permite que el procesamiento sea idempotente: si RabbitMQ reentrega un evento, el Muro lo reconoce y no lo aplica dos veces.

Relación con los requisitos y fallos. Respecto de R3, la separación de colas y la actualización de la proyección antes de emitir `feed.ready` buscan que las alertas lleguen con prioridad. Respecto de R4, la consulta paginada por gRPC y el procesamiento por lotes evitan sobrecargar al servicio con un solo mensaje de gran tamaño. Respecto de F2 (caída del Muro mientras un negocio publica), las colas durables de RabbitMQ conservan los eventos confirmados y el Muro los retoma al recuperarse, aplicando *ACK* manual solo tras persistir y descartando duplicados por `eventId`. Cabe aclarar que estas son medidas de diseño: los tiempos de R3 y la capacidad de R4 no se han comprobado y requerirán pruebas de carga durante la implementación.

4.3. Mensajería directa

El servicio de Mensajería gestiona los chats privados, la presencia de los usuarios y la recuperación de conversaciones. En el diagrama se comunica con el Cliente, PostgreSQL y RabbitMQ mediante cuatro interacciones: I04, I13, I14 e I15.

I04. Comunicación entre Cliente y Mensajería. Para los chats se eligió WebSocket seguro (WSS) sobre TLS/TCP, utilizando la biblioteca `ws`. Esta conexión permite enviar y recibir mensajes sin abrir una petición HTTP por cada intercambio. También sirve para informar cuándo un usuario se conecta o desconecta.

WebSocket se utiliza en aplicaciones de chat en tiempo real y forma parte de las tecnologías estudiadas para sistemas web que requieren baja latencia y múltiples conexiones simultáneas [9], [10].

Cuando el cliente envía un mensaje, el servicio primero debe guardarlo en PostgreSQL. Después devuelve un ACK de aplicación para confirmar que quedó registrado. Esto no significa que el destinatario ya lo recibió o leyó; únicamente confirma el almacenamiento.

I14. Almacenamiento de mensajes en PostgreSQL. Mensajería utiliza el protocolo de PostgreSQL sobre TCP para guardar y consultar las conversaciones. La comunicación es síncrona porque necesita conocer el resultado de la operación antes de confirmar el mensaje al cliente.

Cada mensaje tendría un identificador único y un número de secuencia dentro de su conversación. Esta secuencia se asignaría de forma atómica para evitar que dos mensajes ocupen la misma posición. El identificador también permite reconocer reintentos sin guardar el contenido dos veces.

Si el usuario pierde la conexión, puede regresar y enviar `resume(lastSeq)`, indicando el último mensaje que conoce. A partir de ese dato, Mensajería consulta PostgreSQL y recupera los mensajes que le faltan.

I15. Publicación de mensajes pendientes en RabbitMQ. Cuando el destinatario está desconectado, Mensajería conserva el mensaje en PostgreSQL y publica el evento `dm.pending` en RabbitMQ, utilizando AMQP 0-9-1 sobre TCP. La comunicación es asíncrona porque no necesita esperar a que otro servicio termine de procesar la notificación.

Las comparaciones entre sistemas de colas muestran que la latencia y la cantidad de mensajes procesados cambian según las condiciones de trabajo [11]. Por eso se utiliza RabbitMQ para separar la publicación del evento de su procesamiento posterior, sin asumir que siempre ofrecerá el menor tiempo de respuesta.

También hay que considerar qué ocurre si el mensaje se guarda, pero Mensajería falla antes de publicar el evento. Para ese caso se plantea registrar un evento pendiente en una tabla *outbox*, dentro de la misma transacción que almacena el mensaje. Luego, un proceso separado puede publicarlo en RabbitMQ. Este mecanismo permite coordinar el almacenamiento con el envío de eventos [12].

Se contemplan confirmaciones de publicación y reintentos. Como un evento puede enviarse más de una vez, cada consumidor debe reconocer su identificador para evitar procesarlo repetidamente.

I13. Recepción de eventos desde RabbitMQ. El servicio de Muro publica `feed.ready` cuando termina de preparar una actualización de la línea de tiempo, como se explicó en I12. RabbitMQ entrega ese evento a Mensajería, que puede avisar al usuario mediante WebSocket si mantiene una conexión activa.

La recepción utiliza AMQP 0-9-1 y se propone confirmar cada evento con un ACK manual después de procesarlo. Si Mensajería falla antes de enviar la confirmación, RabbitMQ podría entregar el mismo evento nuevamente. Por eso es necesario identificar los eventos ya atendidos.

La gestión de cargas asíncronas también presenta dificultades en entornos de microservicios [13]. En Vecindario, reconocer los eventos repetidos ayudaría a evitar que el usuario reciba el mismo aviso varias veces.

Requisitos y situaciones de fallo. R2 exige que los mensajes directos lleguen en menos de 500 ms y mantengan su orden. Para ello se utiliza WebSocket, que mantiene la conexión abierta, y PostgreSQL, donde cada mensaje conserva su número de secuencia. El tiempo se comprobaría midiendo desde que un usuario envía un mensaje hasta que lo recibe el destinatario conectado. También habría que revisar el orden de llegada cuando se envían varios mensajes seguidos.

R6 establece una disponibilidad del 99,5 % para Mensajería. Esto significa que el servicio debería permanecer operativo durante casi todo el periodo evaluado. PostgreSQL permite conservar las conversaciones ante un reinicio y WebSocket admite que los clientes vuelvan a conectarse. Sin embargo, estas medidas no evitan todas las interrupciones. Para comprobar el requisito se necesitaría registrar el tiempo de disponibilidad y evaluar mecanismos de recuperación o redundancia.

En F1, si el destinatario está desconectado, el mensaje queda guardado y puede recuperarse cuando vuelva a conectarse. En F3, si Mensajería se reinicia, los clientes pierden su conexión WebSocket y deben restablecerla. Una vez conectados, pueden solicitar los mensajes posteriores a su última secuencia registrada.

Una alternativa sería consultar periódicamente por HTTP si existen mensajes nuevos. El problema es que se realizarían solicitudes incluso cuando no hubiera novedades. Por eso se mantiene WebSocket para el chat y RabbitMQ para los eventos entre servicios.

4.4. Moderación y notificaciones

Esta parte comprende las interacciones I05, I06, I07, I16 e I17. Moderación registra las denuncias y solicita el ocultamiento de publicaciones. Notificaciones mantiene un buzón de avisos para los usuarios. Los mecanismos descritos son propuestas de diseño; todavía no representan funciones implementadas ni resultados comprobados.

I05. Registro de denuncias desde el Cliente. El Cliente envía a Moderación el identificador de la publicación (`postId`), el identificador del denunciante (`reporterId`) y el motivo. Se utiliza una API REST con JSON mediante HTTPS sobre TCP/TLS. El servidor debe comprobar la identidad del denunciante a partir de su sesión, sin confiar únicamente en el identificador recibido.

La comunicación es síncrona porque el Cliente espera una confirmación del registro o un error. Esta respuesta permite informar al usuario si su denuncia fue aceptada, pero no significa que la

publicación ya haya sido ocultada.

I06. Almacenamiento y conteo de denuncias. Moderación guarda las denuncias en PostgreSQL mediante su protocolo nativo sobre TCP. La comunicación es síncrona porque el servicio necesita conocer el resultado de la operación.

Se propone la restricción `UNIQUE(post_id, reporter_id)` para que una persona cuente una sola vez al denunciar una misma publicación. El registro y el conteo deben coordinarse por publicación para evitar resultados incorrectos cuando llegan varias denuncias al mismo tiempo.

Al alcanzar tres denunciantes distintos, se registra una solicitud de ocultamiento pendiente en la misma transacción local. Así, si Moderación se reinicia después de guardar la tercera denuncia, puede recuperar la solicitud y continuar su procesamiento.

I07. Solicitud de ocultamiento mediante gRPC. Después de confirmar la transacción, Moderación solicita a Publicaciones la operación `HidePost(postId, motivo, requestId)`. Se utiliza una llamada RPC unaria mediante gRPC, con Protocol Buffers sobre HTTP/2 y TCP/TLS. Es una comunicación síncrona porque Moderación espera el resultado de cada llamada.

Se elige gRPC para definir claramente los datos de la solicitud y de la respuesta entre ambos servicios. Esto no implica que su uso garantice por sí solo el cumplimiento del tiempo exigido por el escenario.

La llamada tendrá un tiempo de espera limitado. Si ocurre un fallo temporal, la solicitud permanece pendiente y puede reintentarse usando el mismo `requestId`. La operación debe ser idempotente: repetirla mantiene la publicación oculta, sin repetir sus efectos.

Esta precaución es necesaria porque una respuesta podría perderse aunque Publicaciones haya ejecutado el ocultamiento. La solicitud pendiente se marca como completada cuando se confirma el resultado.

Propagación del ocultamiento y requisito R5. Publicaciones guarda el estado oculto del aviso y el evento `post.hidden` en una misma transacción mediante su tabla *outbox* (I08). Después, el evento se publica en RabbitMQ (I09) y el Muro lo consume por una ruta de control separada (I10), actualizando su proyección y descartando copias antiguas de la caché (I18).

Schwarz y colaboradores describen un caso real en el que los servicios guardan eventos en una tabla *outbox* y otro componente los envía al intermediario de mensajería al menos una vez [6]. Este caso respalda el uso del mecanismo para coordinar el almacenamiento con el envío posterior de eventos; los reintentos requieren controlar los posibles duplicados.

El cambio en Publicaciones no aparece de manera instantánea en todos los servicios. Esta actualización progresiva supone consistencia eventual, un aspecto que debe considerarse al coordinar microservicios mediante eventos [5].

R5 exige ocultar el aviso en menos de un minuto después de alcanzar tres denuncias válidas de usuarios distintos. Para comprobarlo, se deberá medir desde el registro de la tercera denuncia hasta que la publicación deje de aparecer en las consultas del Muro. El diseño propone medidas para alcanzar esa meta, pero no garantiza el plazo durante una caída prolongada.

I16. Recepción de eventos en Notificaciones. Notificaciones recibe desde RabbitMQ los eventos `dm.pending`, que informan de mensajes pendientes, y `feed.ready`, que indican una actualización disponible del Muro. Se utiliza AMQP 0-9-1 sobre TCP. La comunicación es asíncrona porque el productor no espera a que Notificaciones termine de procesar cada aviso.

Se propone una cola propia de Notificaciones, durable y con mensajes persistentes. Si Mensajería también necesita recibir `feed.ready`, tendrá otra cola enlazada al mismo evento, de modo que ambos servicios reciban su copia.

Notificaciones guarda el aviso en su buzón y registra su `eventId` en una misma transacción local. Solo después envía el ACK manual a RabbitMQ. Si el servicio falla antes de confirmar el consumo, el evento puede reentregarse. El identificador permite reconocerlo y evitar avisos duplicados.

I17. Consulta del buzón desde el Cliente. El Cliente consulta su buzón mediante una API REST con JSON sobre HTTPS y TCP/TLS. La comunicación es síncrona: el Cliente espera la lista de notificaciones y sus estados de lectura. Se utiliza GET para consultar y una operación de escritura, como PATCH, para marcar una notificación como leída. El servicio debe comprobar que el usuario solo acceda a su propio buzón.

Una notificación de mensaje pendiente no sustituye la entrega del chat. El mensaje original permanece almacenado en Mensajería y se recupera mediante su mecanismo de reconexión. Asimismo, el aviso activo a un cliente conectado se realiza mediante Mensajería por I13 e I04; la consulta del buzón no garantiza por sí sola la entrega inmediata de una alerta.

Fallos, alternativa y limitaciones. Si Notificaciones deja de funcionar, los eventos pendientes pueden conservarse en RabbitMQ y procesarse al recuperar el servicio. Los avisos ya confirmados deben permanecer en el almacenamiento duradero del buzón.

Una alternativa sería llamar directamente a Notificaciones desde los servicios productores. Sin embargo, eso haría que cada llamada dependiera de su disponibilidad. RabbitMQ permite procesar los avisos por separado, a cambio de administrar las colas, los reintentos y los duplicados.

Si falla el Muro, el evento de ocultamiento puede quedar pendiente hasta su recuperación. Por ello, confirmar el cambio en Publicaciones no debe confundirse con haber actualizado ya todas las consultas del Muro.

5. Requisitos del escenario y decisiones de diseño

La siguiente tabla relaciona cada requisito del Anexo A con las interacciones del diagrama y las decisiones de diseño que buscan atenderlo. Se trata de medidas propuestas: ninguna de las cifras del enunciado ha sido comprobada mediante pruebas, por lo que deben entenderse como objetivos de diseño.

ID	Requisito del Anexo A	Evidencia/decisión en el diseño
R1	200 000 usuarios; hasta 40 000 concurrentes	Solo los usuarios conectados mantienen una conexión WebSocket (I04). La consulta del muro lee una proyección ya construida (I03, I18), sin recalcularla en cada petición, y la distribución de avisos se realiza de forma asíncrona mediante RabbitMQ (I09, I10, I12), de modo que no bloquea a los demás servicios. Se propone ejecutar varias instancias de Mensajería y de la API detrás de un balanceador, con presencia de usuarios que expire por tiempo (TTL); estos elementos no aparecen aún en el diagrama. La capacidad para 40 000 conexiones simultáneas requiere pruebas de carga.
R2	Mensajes directos en menos de 500 ms y en orden	La conexión WebSocket permanece abierta (I04), por lo que el servidor puede entregar el mensaje sin esperar una nueva petición [9], [10]. Cada mensaje se guarda en PostgreSQL (I14) con un identificador único y un número de secuencia asignado de forma atómica por conversación; el orden lo define el servidor al aceptar el mensaje. El ACK de aplicación se envía solo después de guardarlo. El tiempo debe medirse desde el envío hasta la recepción por un destinatario conectado.
R3	Muro normal hasta 5 s; alerta en menos de 1 s	Se separan las rutas `feed.normal` y `feed.alert`, con consumidores dedicados (I09, I10), para que un lote de avisos comunes no retrase una alerta. El Muro actualiza su proyección (I18) antes de publicar `feed.ready` (I12), y Mensajería avisa a los clientes conectados mediante I13 e I04. Los plazos de 5 s y 1 s no están comprobados y deben medirse desde la publicación hasta que el aviso aparece en el muro.
R4	Publicación a 30 000 seguidores sin degradar el resto	El Muro consulta a Seguidores mediante gRPC paginado (I11), con `pageToken` y `pageSize`, y procesa los seguidores por lotes en lugar de recibir una lista completa en un solo mensaje. La distribución es asíncrona (I09, I10), con consumidores que pueden escalarse y que reconocen eventos repetidos mediante `eventId` (I18). Se debe verificar mediante pruebas que el servicio no se degrade para el resto de los usuarios.
R5	Tres denuncias; ocultar en menos de 1 minuto	La denuncia llega por REST (I05) y se guarda con la restricción `UNIQUE(post_id, reporter_id)`, de modo que cada persona cuenta una sola vez; el umbral de tres denuncias se evalúa en una transacción (I06). Moderación solicita a Publicaciones `HidePost` mediante gRPC con un `requestId` idempotente (I07). El evento `post.hidden` se registra con *outbox* (I08) y se publica en RabbitMQ (I09) [6]; el Muro lo consume (I10) y actualiza su proyección (I18). El plazo debe medirse desde la tercera denuncia hasta que el aviso deja de aparecer en el muro.

ID	Requisito del Anexo A	Evidencia/decisión en el diseño
R6	Disponibilidad de mensajería: 99,5 %	Las conversaciones se conservan en PostgreSQL (I14) y los clientes pueden reconectarse y recuperar mensajes con `resume(lastSeq)`. Como el muro se alimenta por RabbitMQ con colas durables, una caída del Muro no debería detener el chat, lo que coincide con que el enunciado permite degradar el muro antes que la mensajería. Se proponen réplicas y monitoreo del servicio. Estas medidas no garantizan la disponibilidad: debe registrarse el tiempo operativo real durante un periodo de prueba.

6. Situaciones de fallo

F1: destinatario desconectado. Si una persona envía un mensaje a alguien que está desconectado, Mensajería lo guarda en PostgreSQL antes de confirmar su registro. El mensaje no depende de que el destinatario mantenga abierta una conexión WebSocket.

También se plantea publicar `dm.pending` en RabbitMQ para gestionar la notificación pendiente. El patrón *outbox* permite registrar el evento junto con el mensaje y publicarlo después, incluso si ocurre una interrupción entre ambas operaciones [12].

Cuando el destinatario vuelve a conectarse, puede enviar `resume(lastSeq)` para recuperar los mensajes posteriores al último que conoce. Así, la entrega pendiente se resuelve consultando el historial almacenado.

F2: falla el servicio de Muro. Si el Muro deja de funcionar mientras un negocio publica un aviso, Publicaciones puede guardar el aviso y su evento pendiente en PostgreSQL. Después, un despachador lo publica en RabbitMQ, sin depender de que el Muro esté disponible en ese momento.

Para conservar los eventos durante la interrupción se plantean colas durables y mensajes persistentes. Al recuperarse, el Muro retoma el consumo y actualiza su proyección. El ACK manual se envía después de guardar el cambio; si un evento se entrega nuevamente, su `eventId` permite reconocerlo y evitar que se aplique dos veces.

Este caso muestra por qué es necesario controlar los eventos repetidos y la consistencia entre servicios que procesan información de forma asíncrona [5].

F3: reinicio de Mensajería. Cuando Mensajería se reinicia, las conexiones WebSocket activas se interrumpen. Los clientes deben establecer una nueva conexión para continuar enviando y recibiendo mensajes.

Como las conversaciones se guardan en PostgreSQL, el servicio puede consultar el historial después de recuperarse. Cada cliente indica su última secuencia conocida mediante `resume(lastSeq)`, y Mensajería devuelve los mensajes que faltan.

Los identificadores únicos ayudan a reconocer mensajes reenviados durante la reconexión. Esta recuperación permite continuar las conversaciones, aunque no evita la interrupción momentánea del servicio ni demuestra por sí sola el cumplimiento de la disponibilidad exigida.

7. Alternativas y limitaciones del diseño

Para Vecindario se eligieron distintas tecnologías según el tipo de comunicación. Consultar el Muro no requiere lo mismo que enviar un mensaje privado o distribuir una alerta. Por eso se compararon las opciones utilizadas en el diagrama con otras que podrían resolver las mismas operaciones.

La comparación considera cinco aspectos: comunicación en tiempo real, llamadas entre servicios, distribución de eventos, almacenamiento y consistencia. Las referencias sirven para examinar las diferencias entre tecnologías, no para afirmar que Vecindario ya alcanzó un rendimiento determinado.

Tabla 3: Comparación de tecnologías y alternativas para Vecindario

Aspecto	Elección del equipo	Alternativa	Comparación y decisión
Sockets y tiempo real	WebSocket seguro (WSS) con `ws`	*Long polling*, SSE y Socket.IO	WebSocket permite enviar y recibir información por una misma conexión. SSE se orienta al envío de eventos del servidor al cliente, mientras que *long polling* requiere nuevas solicitudes. Se mantiene WSS para el chat y la presencia. La biblioteca `ws` es una opción más ligera que Socket.IO, aunque exige resolver parte de la reconexión y recuperación en la aplicación [14], [15].
RPC entre servicios	gRPC con HTTP/2 y Protocol Buffers	API REST con HTTPS y JSON	REST facilita las solicitudes habituales del cliente. Para consultar seguidores en I11 se eligió gRPC por su contrato definido y la transmisión de datos binarios. REST sería una alternativa válida, pero no ofrece el mismo contrato tipado de forma nativa. Las diferencias de rendimiento dependen de las operaciones y del entorno evaluado [2], [16].
Mensajería asíncrona	RabbitMQ con AMQP 0-9-1	Apache Kafka	RabbitMQ permite trabajar con colas, confirmaciones y consumidores separados. Kafka ofrece un modelo basado en registros de eventos retenidos, útil cuando se necesita reproducirlos. Se eligió RabbitMQ para la distribución y las notificaciones; la comparación disponible muestra ventajas distintas según la latencia y el volumen procesado [11].

Aspecto	Elección del equipo	Alternativa	Comparación y decisión
Persistencia	PostgreSQL	MongoDB	PostgreSQL permite guardar mensajes, avisos y registros relacionados mediante transacciones. MongoDB ofrece un modelo documental que puede resultar conveniente para información con otra estructura. Las pruebas comparativas no muestran un ganador para todas las operaciones. En Vecindario se mantiene PostgreSQL por las relaciones y transacciones previstas en el diseño [17].
Consistencia de eventos	Patrón *outbox*	Guardar en la BD y publicar directamente	Publicar inmediatamente después de guardar es más sencillo, pero una caída entre ambas acciones puede impedir el envío del evento. El *outbox* registra el evento pendiente dentro de la transacción local y permite publicarlo después. A cambio, necesita un despachador, reintentos y control de duplicados [5], [12].

La comparación muestra que cada tecnología responde a una necesidad distinta. Para Vecindario se eligieron conexiones persistentes para el chat, llamadas síncronas para las consultas y mensajería asíncrona para distribuir eventos. Estas decisiones también tienen sus dificultades: mantener conexiones activas, administrar las colas y controlar los mensajes duplicados.

8. Preguntas críticas

Las preguntas y respuestas que se presentan a continuación son simuladas. La actividad GA-TL-02 no se realizó en clase, sino que fue asignada para desarrollarse de forma remota, por lo que no hubo exposición presencial ni intervención de otros equipos. En consecuencia, el equipo formuló estas preguntas por cuenta propia, a partir de los requisitos (R1–R6) y las situaciones de fallo (F1–F3) del enunciado, con el fin de anticipar los cuestionamientos técnicos más probables sobre el diseño y de preparar su defensa. Ninguna de ellas corresponde a una pregunta efectivamente recibida.

Pregunta 1: ¿Qué ocurre si RabbitMQ reentrega un mensaje al servicio de Muro? ¿No se duplicarían los avisos en el muro?

Respuesta: RabbitMQ garantiza la entrega al menos una vez, no exactamente una vez, por lo que una reentrega es posible, por ejemplo si el Muro falla antes de confirmar el mensaje. Para evitar duplicados, el procesamiento se diseñó de forma idempotente: cada evento incluye un identificador (`eventId`) que el Muro registra en su base de datos PostgreSQL (interacción I18) junto con la proyección. Si el mismo evento llega de nuevo, el Muro detecta que ya fue procesado

y lo descarta. Además, la confirmación (*ACK*) se emite solo después de persistir el cambio, de modo que un fallo a mitad del procesamiento provoca una reentrega y no una pérdida.

Pregunta 2: ¿Qué pasa si Mensajería se reinicia mientras dos usuarios conversan? ¿Cómo se conserva el orden de los mensajes?

Respuesta: Los mensajes aceptados se guardan en PostgreSQL (I14) con una secuencia por conversación. Si el servicio se reinicia, el cliente se reconecta por WebSocket (I04) y utiliza `resume(lastSeq)` para recuperar los mensajes faltantes en orden. Esto permite continuar la conversación, aunque no evita la interrupción momentánea.

Pregunta 3: ¿Cómo evitan que una persona denuncie varias veces una publicación? ¿Qué ocurre si el Muro deja de funcionar?

Respuesta: PostgreSQL impide contar dos veces una denuncia del mismo usuario mediante `UNIQUE(post_id, reporter_id)` (I06). Al llegar a tres denunciantes distintos, Moderación solicita el ocultamiento (I07). Si el Muro falla, el evento `post.hidden` queda pendiente de procesamiento hasta su recuperación, por lo que el plazo de R5 podría verse afectado.

Preguntas formuladas a otros equipos.

No se registran preguntas formuladas a otros equipos, ya que no hubo exposición presencial. Al no existir presentaciones de los demás equipos durante la sesión, no se contó con la oportunidad de plantearles preguntas críticas.

9. Conclusiones

Una de las decisiones más importantes fue evitar que el chat dependiera del funcionamiento del Muro. Los mensajes se almacenan en PostgreSQL y, si Mensajería se reinicia, el cliente puede recuperar los que faltan mediante `resume(lastSeq)`. Así, una desconexión no implica perder los mensajes que ya fueron guardados.

En la distribución de avisos encontramos otra dificultad: un evento puede llegar repetido o quedar pendiente durante una caída. Por eso se plantearon RabbitMQ, el patrón *outbox* y el control de duplicados. No basta con enviar un evento; también hay que comprobar que se procese correctamente.

El diseño propone cómo responder a estos problemas, pero todavía no demuestra que se cumplan los tiempos exigidos. La entrega de mensajes en menos de 500 ms, el ocultamiento de publicaciones en un minuto y la disponibilidad del 99,5 % necesitan pruebas para comprobarse.

10. Declaración de uso de inteligencia artificial

En el desarrollo de este trabajo se utilizó **ChatGPT (OpenAI, GPT-6)** como herramienta de apoyo para interpretar la consigna, organizar la estructura del documento en LaTeX y repartir el trabajo entre los integrantes. **Claude (Anthropic)** se empleó como apoyo en la escritura en LaTeX y en la corrección de gramática y ortografía.

Referencias

[1] S. D. Meglio y L. L. L. Starace, «Evaluating Performance and Resource Consumption of REST Frameworks and Execution Environments: Insights and Guidelines for Developers and Companies,» *IEEE Access*, vol. 12, págs. 161 649-161 669, 2024. DOI: 10.1109/ACCESS.2024.3489892

[2] J. Kazanavičius y D. Mažeika, «The Evaluation of Microservice Communication While Decomposing Monoliths,» *Computing and Informatics*, vol. 42, n.º 1, págs. 1-36, 2023. DOI: 10.31577/cai_2023_1_1

[3] Y. Chen et al., «Understanding the Security Risks of Websites Using Cloud Storage for Direct User File Uploads,» *IEEE Transactions on Information Forensics and Security*, vol. 20, págs. 2677-2692, 2025. DOI: 10.1109/TIFS.2025.3544082

[4] Y. Arhrib, F. Alvarez-Taboada y H. Boulaassal, «Serverless 3D Reconstruction and Spatial Anchoring for Cloud-Native Infrastructure Inspection,» *Buildings*, vol. 16, n.º 12, 2026, **artno** 2433. DOI: 10.3390/buildings16122433

[5] J. L. Bação y S. Guerreiro, «Consistency Challenges in Event-Driven Microservices: A Literature Review on Data, Process, and System Evolution,» *Computing*, vol. 108, 2026, **artno** 110. DOI: 10.1007/s00607-026-01701-5

[6] G.-D. Schwarz, F. Quast y D. Riehle, «Ensuring Syntactic Interoperability Using Consumer-Driven Contract Testing,» *Software Testing, Verification and Reliability*, vol. 35, n.º 5, 2025, **artno** e70006. DOI: 10.1002/stvr.70006

[7] G. Fu, Y. Zhang y G. Yu, «A Fair Comparison of Message Queuing Systems,» *IEEE Access*, vol. 9, págs. 421-432, 2021. DOI: 10.1109/ACCESS.2020.3046503

[8] T. Vairam, Vishal K, Kavin T, Sengathirsoorian E. T. y Murugavel E, «An Empirical Evaluation of gRPC and REST Communication Patterns in Microservice Architectures,» en *2026 Second International Conference on Intelligent Systems for Communication, IoT and Security (ICISCoIS)*, 2026, págs. 1-6. DOI: 10.1109/ICISCoIS62701.2026.11448051

[9] G. Gangadevi, C. Agrawal, J. Chouhan y G. Verma, «End-To-End Development of a Robust Live Chat System for Real-Time Communication,» en *Proceedings of the 4th International Conference on Information Technology, Civil Innovation, Science, and Management (ICITSM 2025), Part I*, EAI, 2025. DOI: 10.4108/eai.28-4-2025.2357824

[10] J. M. Díaz-Gómez, E. Quiceno-Rua y C. D. Correa-Álvarez, «Protocols, Reactive Architectures, and Computing Platforms for Low-Latency, High-Concurrency Web Applications: A Systematic Literature Review,» *Future Internet*, vol. 18, n.º 5, 2026, **artno** 254. DOI: 10.3390/fi18050254

[11] R. Maharjan, M. S. H. Chy, M. A. Arju y T. Cerny, «Benchmarking Message Queues,» *Telecom*, vol. 4, n.º 2, págs. 298-312, 2023. DOI: 10.3390/telecom4020018

[12] N. Gupta, «The Art of Reliability and Asynchrony: Building Robust Distributed System Communication via Outbox,» *International Journal of Engineering and Technology*, vol. 17, n.º 4, págs. 199-202, 2025. DOI: 10.7763/IJET.2025.V17.1327

[13] C. Batista, F. Morais, E. Cavalcante, T. Batista, B. Proença y W. B. Rodrigues Cavalcante, «Managing Asynchronous Workloads in a Multi-Tenant Microservice Enterprise Environment,» *Software: Practice and Experience*, vol. 54, n.º 2, págs. 334-359, 2024. DOI: 10.1002/spe.3278

[14] K. J. Gracia Orejuela, M. B. Gorozabel Bazurto y W. G. Chango Sailema, «Long polling, websockets y server-sent events: comunicación para el envío de datos en tiempo real,» *Mikarimin. Revista Científica Multidisciplinaria*, vol. 10, n.º 3, págs. 101-120, 2024. DOI: 10.61154/mrcm.v10i3.3272

[15] L. Fernando y M. M. Engel, «Comparative Performance Benchmarking of WebSocket Libraries on Node.js and Golang,» *Sinkron: Jurnal dan Penelitian Teknik Informatika*, vol. 9, n.º 4, págs. 2051-2060, 2025. DOI: 10.33395/sinkron.v9i4.15266

[16] Ł. Kamiński, M. Kozłowski, D. Sporysz, K. Wolska, P. Zaniewski y R. Roszczyk, «Comparative Review of Selected Internet Communication Protocols,» *Foundations of Computing and Decision Sciences*, vol. 48, n.º 1, págs. 39-56, 2023. DOI: 10.2478/fcds-2023-0003

[17] J. Olszak y M. Skublewska-Paszkowska, «The Impact of Relational and Non-Relational Databases on Application Performance,» *Journal of Computer Sciences Institute*, vol. 35, págs. 184-190, 2025. DOI: 10.35784/jcsi.7260